

VULNERABILITY ASSESSMENT AND PENETRATION TESTING REPORT

FOR

<https://testfire.net/>

FROM

SAUDA MOMIN

CONTENTS

Sr.no	Content	Page number
1.	Executive summary Summary Objective Report	3
	Tabular summary	4
3	Graphical Summary	5
4	Technical report	
4.1	SQL Injection - Login bypass	6-7
4.2	Sensitive data exposure - unrestricted access to internal content	8-9
4.3	Clear text credentials	10-11
4.4	Exposing development or debugging tools	12-13
4.5	Cryptographic failure	14-15
4.6	Insecure direct object	16-17
4.7	Security misconfiguration - Use of Default Credentials and Weak Authentication Hardening	18-20
4.8	Login brute force of username and password	21-23
4.9	Insecure design - Duplicate fund transfer	24-26
4.10	Vulnerable and outdated component	27-28
4.11	Session cookie exposure via document.cookie	29-30
4.12	Reflected cross site scripting	31-32
4.13	Improper input validation	33-35
4.14	Security misconfiguration - missing security header	36-37
4.15	Displaying internal server error	38-39
5	Conclusion	40

SUMMARY

1.1 EXECUTIVE SUMMARY

INSTITUTE OF INFORMATION SECURITY (IIS) has assigned the task of vulnerability assessment and penetration testing (VAPT) on testfire.net.

This VAPT was performed on 26th june 2025. The detailed report and our findings are described below.

1.2 OBJECTIVE

The objective of this test was to determine security vulnerabilities in the web server configuration and website running on the server. The tests were carried out assuming the identity of an attacker or with malicious intent. At the same time due care was taken not to harm the web server.

1.3 SCOPE

The scope of our work was limited to only testfire.net. The vulnerabilities and our findings are described below.

TABULAR SUMMARY

The following tables summarize the vulnerability assessment of the server.

Category	Description
No. of live host	1
No. vulnerabilities	15
0No. of critical vulnerabilities	2
No. of high vulnerabilities	8
No. of medium vulnerabilities	3
No. of low vulnerabilities	2

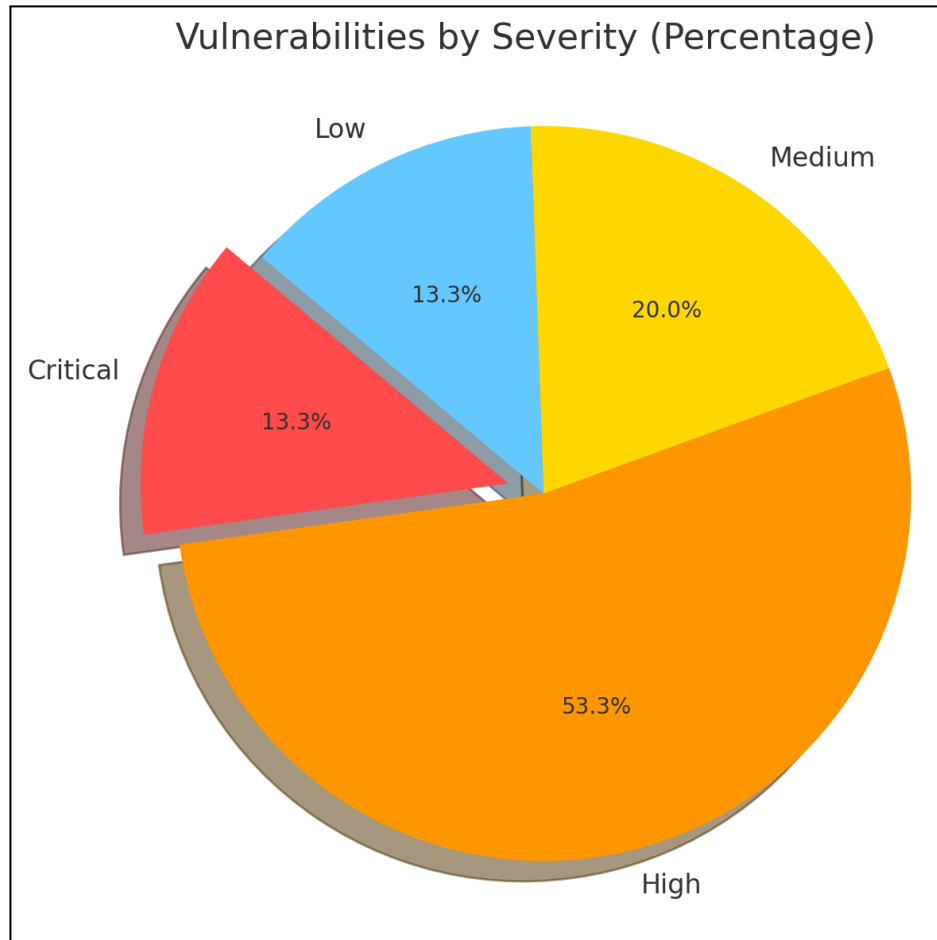
Fig 2.1

SrNo.	Vulnerability name	Severity	Ease of Exploitation
1	SQL Injection - Log in bypass	Critical	Easy
2	Sensitive data exposure - unrestricted access to internal content	Critical	Easy
3	Clear text credentials	High	Moderate
4	Exposing development or debugging tools	High	Easy
5	Cryptographic failure	High	Moderate
6	Insecure direct object	High	Easy
7	Security misconfiguration - Use of Default Credentials and Weak Authentication Hardening	High	Easy
8	Log in brute force of username and password	High	Easy
9	Insecure design - Duplicate fund transfer	High	Moderate
10	Vulnerable and outdated component	High	Moderate
11	Session cookie exposure via document.cookie	High	Easy
12	Reflected cross site scripting	Medium	Easy
13	Improper input validation	Medium	Easy
14	Security misconfiguration - missing security header	Medium	Easy
15	Displaying internal server error	Low	Easy

Fig 2.2

GRAPHICAL SUMMARY

The following pie chart graphically summaries the vulnerability assessment



3.3 – **Pie chart** showing the percentage distribution of vulnerabilities by severity

1. SQL INJECTION - Login Bypass

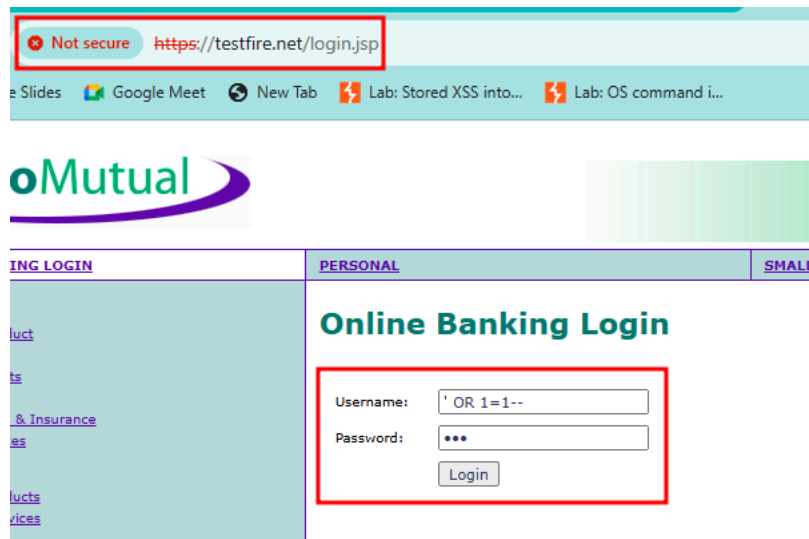
The SQL Injection occurs when data input by a user is interpreted as a SQL command rather than as normal data by the backend database.

Affect on

Infected URL	https://testfire.net/login.jsp
Infected Parameter	Username
Parameter type	Post
Attack vector	Uid = ' or 1=1-- -

Analysis

The uid parameter in the login form is directly passed into a SQL query without input sanitization or parameterization. This allows attackers to manipulate the query logic and bypass authentication. By injecting a tautology like ' OR 1=1--, the application logs in the user without valid credentials, indicating a critical SQL Injection vulnerability.



Hello Admin User

Welcome to Altoro Mutual Online.

View Account Details:

800000 Corporate ▼

GO

Congratulations!

You have been pre-approved for an Altoro Gold Visa with a credit limit of \$10000!

Click [Here](#) to apply.

Impact

- Authentication bypass: The attacker can log in without valid credentials.
- Privilege escalation: If the query returns admin-level users, full access may be granted.
- Unauthorized access to user accounts.
- Complete compromise of the application if admin panel is accessible.
- Data leakage or modification depending on user rights after login

Root Cause

- Direct inclusion of unsanitized user input in SQL query
- Lack of parameterized queries or input validation
- Failure to use secure coding practices for database access

Mitigation

- Use parameterized queries / prepared statements
- Validate and sanitize all user inputs
- Apply least privilege principle to database users
- Use ORM frameworks to manage database access securely
- Deploy Web Application Firewall (WAF) to block SQL patterns
- Conduct regular security assessments and code reviews

2. Sensitive Data Exposure - Unrestricted Access to Internal Content

Affect on

Infected URL	https://testfire.net/index.jsp?content=inside_jobs.htm
Infected Parameter	Content
Parameter type	URL query string
Attack vector	An attacker accesses the inside_jobs.htm page via URL parameter manipulation. The server includes internal content without access control, exposing potentially sensitive internal job details or user information that can aid further attacks.

Analysis

The application exposes internal job listings (e.g., Operational Risk Manager, Mortgage Lending Account Executive) through a URL parameter without access restriction. This page appears to be intended for internal use ("INSIDE ALTORO MUTUAL") but is accessible to any user without authentication. This represents a Broken Access Control vulnerability and may assist attackers in understanding the organization's internal structure, making social engineering or phishing more targeted and effective.

testfire.net/index.jsp?content=inside_jobs.htm

Sign Off | Contact Us | Feedback | Search

Current Job Openings

We update our job database daily so that you can find the most up-to-date career opportunities within Altoro Mutual.

Group	Date Posted	Title
Administration	Oct-23-2006	Executive Assistant
Consumer Banking	Oct-19-2006	Teller
Customer Service	Oct-26-2006	Customer Service Representative
Marketing	Oct-25-2006	Loyalty Marketing Program Manager
Risk Management	Oct-17-2006	Operational Risk Manager
Sales	Oct-24-2006	Mortgage Lending Account Executive

Altoro Mutual and its affiliates recruit and hire qualified candidates without regard to race, religion, color sex, sexual orientation, age, national origin, ancestry, citizenship, veteran or disability status or any factor prohibited by law, and as such affirms in policy and practice to support and promote the concept of equal employment opportunity and affirmative action, in accordance with all applicable federal, state and municipal laws. Candidates must possess the right to work in the United States, as it is not the practice of Altoro Mutual to sponsor individuals for work visas.

Server Status Check | REST API | © 2025 Altoro Mutual, Inc. *This web application is open source! Get your copy from GitHub and take advantage of advanced features*

Impact

- Discloses internal job roles and organizational structure to unauthenticated users.
- Enables attackers to craft targeted phishing or social engineering attacks using role-specific information.
- Reduces overall security posture by unintentionally exposing internal content.
- May lead to reputational risk if attackers use this data to impersonate internal departments or staff.
- Provides attackers with insights into sensitive business functions and hierarchy.

Root Cause

- Lack of access control checks on internal content pages.
- Use of URL parameters to directly include internal HTML files without proper validation.
- No authentication required to view potentially internal or restricted content.
- Poor separation between publicly accessible and internal-only resources.

Mitigation

- Apply strict access control to any internal or sensitive content pages.
- Avoid including internal pages through client-controllable parameters like ?content=....
- Implement a whitelist of allowed content pages for dynamic loading.
- Require authentication and proper authorization checks for access to internal resources.
- Regularly review and audit URLs and parameters that expose server-side files or views.

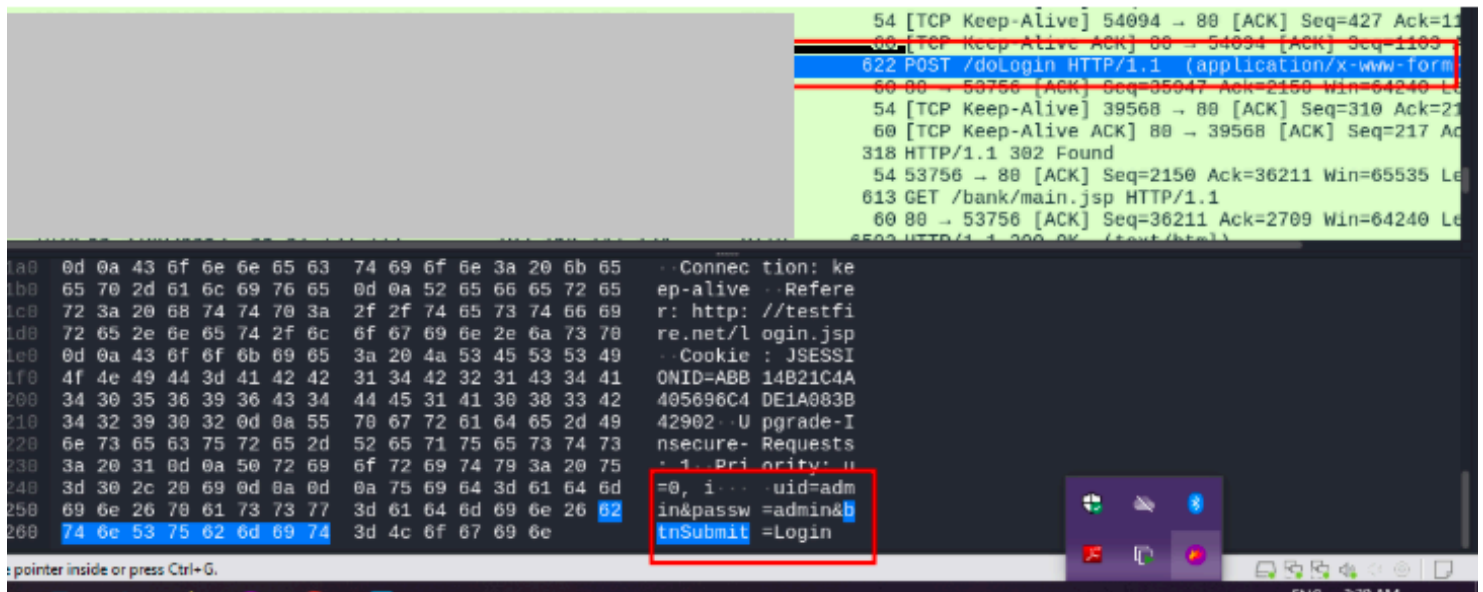
3. Clear Text Credentials

Affect on

Infected URL	https://testfire.net/login.jsp
Infected Parameter	Username,passwd
Parameter type	Post
Attack vector	User credentials are transmitted over HTTP or mixed content, allowing an attacker to intercept username and password via network sniffing (Man-in-the-Middle attack).

Analysis

During the login process on <http://testfire.net/login.jsp>, network traffic was captured using Wireshark for inspection. Upon analyzing the HTTP POST request, it was observed that the username and password were transmitted in clear text over an unencrypted channel.



Impact:

- Exposure of sensitive login credentials over the network.
- Enables attackers to perform Man-in-the-Middle (MitM) attacks to capture and reuse credentials.
- Leads to unauthorized access, identity theft, and potential lateral movement within systems.

Root Cause:

- Application transmits login credentials over unencrypted HTTP.
- Absence of TLS/SSL on the login page and form submission.
- No enforcement of secure communication protocols such as HTTPS or HSTS.

Mitigation:

- Implement HTTPS across all pages that handle sensitive data.
- Configure web server to enforce TLS 1.2 or higher.
- Redirect all HTTP traffic to HTTPS using 301 permanent redirects.
- Enable HTTP Strict Transport Security (HSTS) headers.
- Periodically audit application traffic to ensure all data in transit is encrypted.

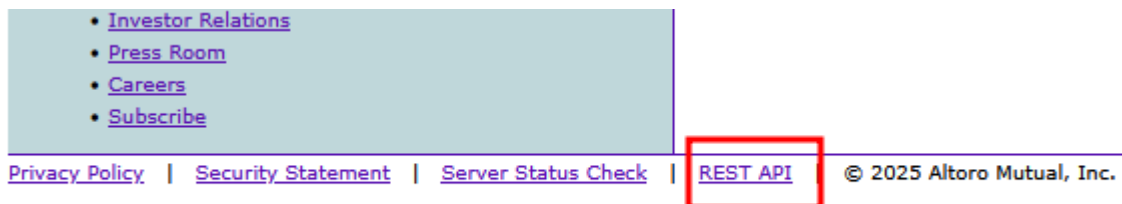
4. Security misconfiguration - Exposing development or debugging tools (e.g., Swagger UI) in a production environment without authentication

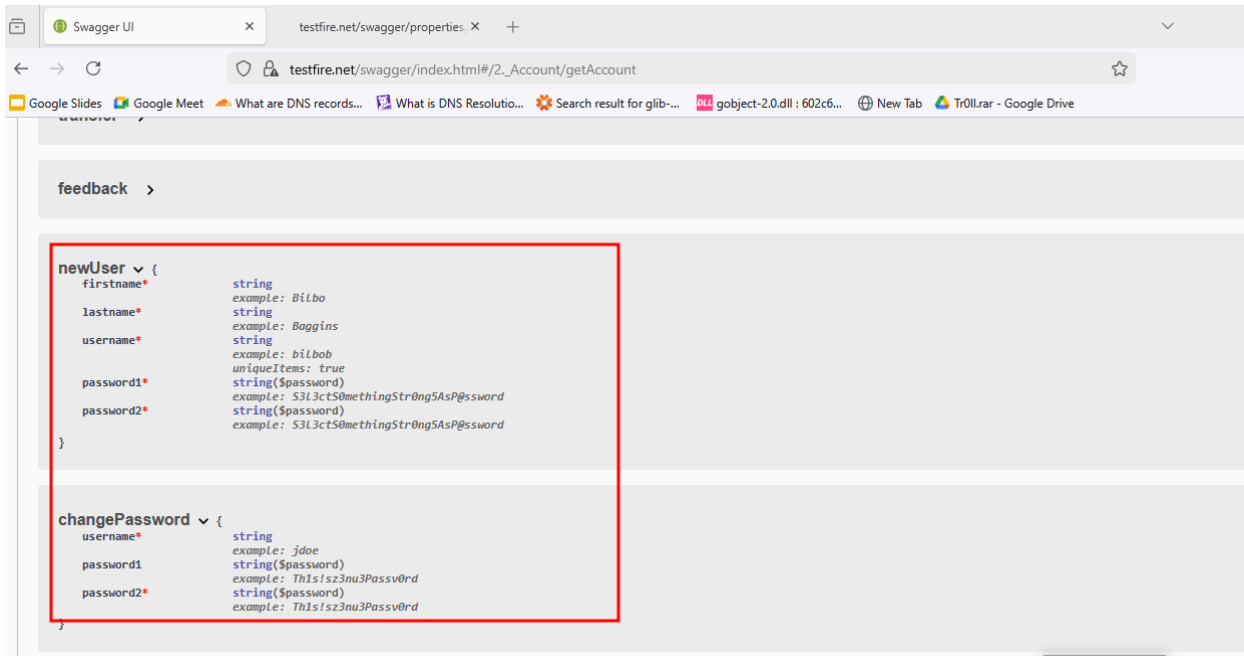
Affect On

Infected URL	http://testfire.net/swagger/index.html
Infected Parameter	N/A (data is exposed in Swagger UI itself)
Parameter type	Page content
Attack vector	An unauthenticated attacker visits the exposed Swagger documentation endpoint. The page reveals real usernames and account numbers as example values in the API documentation. This information can be used to perform further attacks such as credential stuffing, account enumeration, or targeted brute-force attacks.

Analysis

The Swagger documentation at <http://testfire.net/swagger/index.html> is publicly accessible and reveals sensitive information such as usernames and account numbers. This is a result of poor deployment practices and misconfiguration. An attacker can use this information to perform targeted attacks like account enumeration, brute force, or credential stuffing. Since this exposure exists in a production environment, it falls under OWASP A05:2021 – Security Misconfiguration.





Impact

- Leaks sensitive information such as usernames and account numbers.
- Assists attackers in crafting targeted attacks like brute force or credential stuffing.
- Increases the risk of user enumeration and role escalation.
- Reveals internal API structure, making exploitation easier.
- Weakens the overall security posture of the application.

Root Cause

- Swagger UI is exposed in a production environment without authentication.
- No access control implemented for developer tools or documentation.
- Example data in API documentation contains real or realistic user information.
- Failure to disable or restrict internal tools before deploying publicly.

Mitigation

- Disable Swagger and similar developer documentation tools in production environments.
- Protect internal documentation with authentication and access controls.
- Sanitize or remove example data that includes sensitive information.
- Review deployment configurations to ensure no debug or test tools are exposed publicly.
- Conduct regular security reviews to identify exposed endpoints and documentation.

5. Cryptographic failure - Predictable session ID

Affect on

Infected URL	https://testfire.net/login.js
Infected Parameter	Session cookies
Parameter type	Set-Cookie Header (After POST login)
Attack vector	Static or predictable session

Analysis

After logging in multiple times, the session cookie (JSESSIONID) showed similar values, indicating possible predictability. This suggests that the server either reuses tokens or generates them with weak randomness. A predictable session ID allows an attacker to hijack another user’s session by guessing or reusing the token.

PERSONAL

SMALL BUSINESS

INSIDE ALTORO MUTUAL

Summary

Transactions

ds

Articles

ite Language

Hello Admin User

Welcome to Altoro Mutual Online.

View Account Details: 800000 Corporate GO

Congratulations!

You have been pre-approved for an Altoro Gold Visa with a credit limit of \$10000!

Click [Here](#) to apply.

Console

Debugger

Network

Style Editor

Performance

Memory

Storage

Accessibility

Application

Filter Items

Name	Value	Domain	Path	Expires / Max-Age	Size	HttpOnly	Secure	SameSite	Last Accessed	Partition Key
AltoroAcc...	ODAwMDAwfkNvcnBv...	testfire.net	/	Session	90	false	false	None	Thu, 26 Jun 2025 06...	
JSESSION...	9030446823C9DFCF1D...	testfire.net	/	Session	42	true	true	None	Thu, 26 Jun 2025 06...	

JSESSIONID: "9030446823C9DFC...C2586A06DA242A"

Created: Thu, 26 Jun 2025 05:02:53 GMT

Domain: "testfire.net"

Age: "Session"

Thu, 26 Jun 2025 06:48:20 GMT

AltoroAcc...	ODAwMDAwfkNvcnBv...	testfire.net	/	Session	90	false	false	None	Thu, 26 Jun 2025 06...
JSESSION...	9030446823C9DFCF1D...	testfire.net	/	Session	42	true	true	None	Thu, 26 Jun 2025 06...

Impact

- Attackers can guess or predict session tokens.
- Allows session hijacking and impersonation of legitimate users.
- May lead to unauthorized access, data leakage, or privilege escalation.
- Compromises the integrity and confidentiality of user sessions.

Root Cause

- Use of weak or predictable algorithms for generating session IDs.
- Static or sequential session IDs reused across logins.
- Insecure or default configurations that do not regenerate session tokens after login or logout.
- Insufficient randomness or entropy in session generation logic.

Mitigations

- Generate session tokens using strong cryptographic algorithms with high entropy (e.g., 128-bit secure random values).
- Always regenerate the session ID after successful login and logout.
- Store session identifiers in secure, HTTP-only cookies.
- Set the Secure and SameSite cookie flags.
- Avoid exposing session IDs in URLs (use only in cookies).
- Conduct regular security testing (e.g., session token entropy analysis via tools like Burp Sequencer).

6. Insecure Direct Object Reference - IDOR

Affect On

Infected URL	https://demo.testfire.net/bank/accountDetail?account=800002
Infected Parameter	Account
Parameter type	GET
Attack vector	User modifies the account number in the URL to access other users' account details without authorization.

Analysis

The app fails to verify ownership of the account parameter, allowing users to access others' data by simply changing the ID in the URL.

testfire.net/bank/showAccount?listAccounts=800000

What are DNS records... What is DNS Resolutio... Search result for glib-... gobject-2.0.dll : 602c6... New Tab Tr0ll.rar - Google Drive

Sign Off | Contact Us | Feedback

tual

PERSONAL SMALL BUSINESS INSIDE AL

Account History - 800000 Corporate

Balance Detail	
800000 Corporate ▼ Select Account	Amount
Ending balance as of 6/26/25 3:07 AM	- \$118446744077952240000.00
Available balance	- \$118446744077952240000.00

testfire.net/bank/showAccount?listAccounts=800001

Google Meet What are DNS records... What is DNS Resolutio... Search result for glib-... gobject-2.0.dll : 602c6... New Tab Tröll.rar

Sign Off | Conta

Mutual

PERSONAL SMALL BUSINESS

Summary Transactions Articles Language

Account History - 800001 Checking

Balance Detail	
800001 Checking Select Account	Amount
Ending balance as of 6/26/25 3:08 AM	\$236893488156009170000.00
Available balance	\$236893488156009170000.00

10 Most Recent Transactions		
Date	Description	Amount
2025-06-26	Deposit	\$1234.00

Impacts:

- Unauthorized access to other users' bank account details
- Exposure of sensitive financial information
- Violation of data privacy and security policies
- Possibility of horizontal privilege escalation
- Increased risk of further attacks like data theft or fraud

Root Cause:

- Lack of proper access control checks on the server
- Direct use of user-controlled parameters (e.g., account numbers) without validation
- Predictable and incrementing account identifiers
- No session-to-resource ownership verification

Mitigation:

- Implement server-side access control to verify resource ownership
- Use indirect object references (e.g., UUIDs or mapping tokens) instead of raw IDs
- Enforce access validation on every sensitive object access request
- Regularly test for IDOR using security tools like Burp Suite or OWASP ZAP
- Follow secure coding practices and implement security in the design phase

7. Security Misconfiguration - Use of Default Credentials and Weak Authentication Hardening

Affect on

Infected URL	http://testfire.net/login.jsp
Infected Parameter	username,password
Parameter type	post
Attack vector	Default credentials (admin/admin) allow full admin access

Analysis

The application allows login using default credentials admin/admin, granting full administrative access. This is a critical security misconfiguration that exposes the system to unauthorized access and privilege escalation. The lack of password complexity enforcement, absence of first-time password change, and no brute-force protection indicate poor authentication practices. This vulnerability can be easily exploited and could lead to complete compromise of the application in a real-world scenario.



ONLINE BANKING LOGIN

PERSONAL

SMALL BUSINESS

Online Banking Login

Username:

Password:

PERSONAL

- [Deposit Product](#)
- [Checking](#)
- [Loan Products](#)
- [Cards](#)
- [Investments & Insurance](#)
- [Other Services](#)

SMALL BUSINESS

- [Deposit Products](#)
- [Lending Services](#)
- [Cards](#)
- [Insurance](#)
- [Retirement](#)
- [Other Services](#)

INSIDE ALTORO MUTUAL

- [About Us](#)
- [Contact Us](#)
- [Locations](#)
- [Investor Relations](#)
- [Press Room](#)
- [Careers](#)
- [Subscribe](#)

MY ACCOUNT

PERSONAL

SMALL BUSINESS

Hello Admin User

Welcome to Altoro Mutual Online.

View Account Details:

Congratulations!

You have been pre-approved for an Altoro Gold Visa with a credit limit of \$10000!

Click [Here](#) to apply.

I WANT TO ...

- [View Account Summary](#)
- [View Recent Transactions](#)
- [Transfer Funds](#)
- [Search News Articles](#)
- [Customize Site Language](#)

ADMINISTRATION

- [Edit Users](#)

Impact

- Unauthorized access to the application as an administrator.
- Full access to sensitive functions such as user management, financial data, and customer details.
- Enables privilege escalation from unauthenticated user to administrator.
- Can be exploited by automated scripts, bots, or malicious actors without technical knowledge.
- In a real-world scenario, could result in complete system compromise, regulatory violations (e.g., GDPR), and reputational damage.

Root Cause

- Use of default credentials (admin/admin) which are commonly known and widely published.
- No enforcement of password change upon first login.
- Absence of account hardening practices, such as account lockout, brute-force protection, or credential rotation.
- Misconfigured authentication system allowing critical roles to use insecure credentials without logging or alerts.

Mitigation

- Immediately disable or change default credentials on all environments (development, staging, production).
- Enforce a strong password policy:
 1. Minimum length of 12 characters.
 2. Require a mix of uppercase, lowercase, numbers, and special characters.
- Require users to change default passwords on first login.
- Implement account lockout after repeated failed login attempts.
- Regularly audit user accounts for weak or default passwords.
- Log and alert on any login attempt using default or common credentials (e.g., admin/admin, root/root).

8. Login Brute Force of Username and Password

Affect On

Infected URL	http://testfire.net/login.jsp
Infected Parameter	Uid, passwd
Parameter type	Post
Attack vector	An attacker can use automated tools like Burp Suite Intruder or Hydra to send multiple login requests with different username/password combinations. The server does not implement account lockout, CAPTCHA, or rate limiting, allowing unlimited guesses and making brute force attacks feasible.

Analysis

The login page at <http://testfire.net/login.jsp> does not implement any protection against repeated login attempts. Using tools like Burp Suite Intruder, an attacker can automate multiple password guesses for a known username without being blocked or rate-limited. This makes the application vulnerable to brute-force attacks and unauthorized access.

Dashboard Target Proxy Intruder Repeater Collaborator Sequencer

1 2 3 x +

Send Cancel < >

Request

Pretty Raw Hex

```
1 POST /doLogin HTTP/1.1
2 Host: testfire.net
3 Cookie: JSESSIONID=14CE61153D6552233EEB203CF03FBE04;
4 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:140.0)
5 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
6 Accept-Language: en-US,en;q=0.5
7 Accept-Encoding: gzip, deflate, br
8 Content-Type: application/x-www-form-urlencoded
9 Content-Length: 37
10 Origin: https://testfire.net
11 Referer: https://testfire.net/login.jsp
12 Upgrade-Insecure-Requests: 1
13 Sec-Fetch-Dest: document
14 Sec-Fetch-Mode: navigate
15 Sec-Fetch-Site: same-origin
16 Sec-Fetch-User: ?1
17 Priority: u=0, i
18 Te: trailers
19 Connection: keep-alive
20
21 uid=admin&passw=admin&btnSubmit=Login
```

Payloads

Payload position: All payload positions

Payload type: Simple list

Payload count: 10

Request count: 10

Payload configuration

This payload type lets you configure a simple list of strings that are used as payloads.

Paste	admin
Load...	user
Remove	test
Clear	root
Deduplicate	toor
	daemon
	dwwa
	admin@123
	admin123
Add	Enter a new item

Add from list... [Pro version only]

Payload processing

Impact

- Allows attackers to guess valid username-password combinations through repeated login attempts.
- Can result in unauthorized access to user or admin accounts.
- May lead to data exposure, account takeover, and privilege escalation.
- Increases risk of automated attacks such as credential stuffing.
- Weakens overall authentication security of the application.

Root Cause

- No account lockout mechanism after multiple failed login attempts.
- Absence of CAPTCHA or human verification.
- No rate limiting or throttling implemented on login requests.
- Server accepts unlimited and rapid login attempts without triggering any alerts or blocks.

Mitigation

- Implement account lockout or temporary suspension after 5–10 failed login attempts.
- Add CAPTCHA or reCAPTCHA to the login form to prevent automated submissions.
- Apply rate limiting per IP or per user to slow down repeated login attempts.
- Monitor and log failed login attempts, and trigger alerts for suspicious patterns.
- Use multi-factor authentication (MFA) to reduce the impact of guessed credentials.

9. Insecure design - duplicate fund transfer

Affect on

Infected URL	http://testfire.net/secure/transferFunds.jsp
Infected Parameter	fromAccount, toAccount, amount
Parameter type	post
Attack vector	Sending same fund transfer request twice via burp repeater

Analysis

The application accepts the same fund transfer request multiple times without validating transaction uniqueness. This indicates a flaw in business logic where the backend fails to detect or block duplicate payments. As a result, an attacker or user can exploit this to initiate multiple unauthorized transactions, leading to financial fraud, double spending, or abuse of payment workflows. This is a classic example of Insecure Design (OWASP A04:2021) due to the lack of defensive controls in the transaction process.

Request

Pretty

Raw

Hex



```
1 POST /bank/doTransfer HTTP/1.1
2 Host: testfire.net
3 Cookie: JSESSIONID=6EF18290E2D3998E24255557C4E1AF51;
  AltoroAccounts=
  ODAwMDAwfkNvcnBvcnFOZX4tNS44NTQ1NDUONTQ1NDUONTVFMjk2fDgwMDAwMX5DaG
  Vja2luZ341Ljg1NDUONTQ1NDUONTQ1NUUyOTZ8
4 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:140.0)
  Gecko/20100101 Firefox/140.0
5 Accept:
  text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
6 Accept-Language: en-US,en;q=0.5
7 Accept-Encoding: gzip, deflate, br
8 Content-Type: application/x-www-form-urlencoded
9 Content-Length: 78
10 Origin: https://testfire.net
11 Referer: https://testfire.net/bank/transfer.jsp
12 Upgrade-Insecure-Requests: 1
13 Sec-Fetch-Dest: document
14 Sec-Fetch-Mode: navigate
15 Sec-Fetch-Site: same-origin
16 Sec-Fetch-User: ?1
17 Priority: u=0, i
18 Te: trailers
19 Connection: keep-alive
20
21 fromAccount=800000&toAccount=800001&transferAmount=500&transfer=
  Transfer+Money
```

I WANT TO ...

- [View Account Summary](#)
- [View Recent Transactions](#)
- [Transfer Funds](#)
- [Search News Articles](#)
- [Customize Site Language](#)

ADMINISTRATION

- [Edit Users](#)

Recent Transactions

After

yyyy-mm-dd

Before

yyyy-mm-dd

Transaction ID	Transaction Time	Account ID	Action	Amount
21212	2025-06-26 08:21	800001	Deposit	
21211	2025-06-26 08:21	800000	Withdrawal	
21210	2025-06-26 08:21	800001	Deposit	
21209	2025-06-26 08:21	800000	Withdrawal	

Impact

- Duplicate payments can lead to financial losses for the institution or users.
- Attackers can exploit the flaw to perform unauthorized repeated transfers.
- Trust in the financial system is reduced due to inconsistent transaction behavior.
- Increases risk of fraud, abuse, and regulatory non-compliance.
- May result in chargebacks, accounting issues, or litigation in real-world systems.

Root Cause

- Lack of server-side validation to detect and prevent repeated transaction requests.
- No implementation of unique transaction identifiers or anti-replay tokens.
- Absence of transaction state tracking (e.g., marking a transaction as "completed").
- Over-reliance on client-side form behavior or UI disablement to prevent resubmission.

Mitigation

- Implement transaction IDs that are validated on the server to ensure one-time use.
- Enforce backend checks to reject duplicate or replayed requests.
- Track the state of each transaction to prevent re-processing.
- Introduce anti-replay tokens (nonce) for each transaction form.
- Log and monitor duplicate payment attempts for anomaly detection and alerting

10. Vulnerable and Outdated Components

Affect on

Infected URL	https://demo.testfire.net/
Infected Parameter	HTTP response headers
Parameter type	Server configuration
Attack vector	Server does not implement required security headers (X-Frame-Options, HSTS, X-Content-Type-Options)

Analysis

The server runs on an outdated platform (Apache-Coyote/1.1) and lacks critical security headers (X-Frame-Options, HSTS, X-Content-Type-Options). It also exposes unsafe HTTP methods like PUT and DELETE. These misconfigurations make the application vulnerable to clickjacking, insecure content handling, and potential file upload or deletion attacks. This indicates poor maintenance and hardening, aligning with OWASP A06:2021 – Vulnerable and Outdated Components.

```
(root@kali)-[/home/kali]
└─$ nikto -h https://demo.testfire.net/
- Nikto v2.5.0

+ Target IP: 65.61.137.117
+ Target Hostname: demo.testfire.net
+ Target Port: 443

+ SSL Info: Subject: /CN=demo.testfire.net
            Ciphers: ECDHE-RSA-AES256-GCM-SHA384
            Issuer: /C=GB/ST=Greater Manchester/L=Salford/O=Sectigo Limited/CN=Sectigo RSA Domain Validation Secure Server CA
+ Start Time: 2025-06-26 07:31:54 (GMT-4)

+ Server: Apache-Coyote/1.1
+ /: The anti-clickjacking X-Frame-Options header is not present. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/X-Frame-Options
+ /: The site uses TLS and the Strict-Transport-Security HTTP header is not defined. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Strict-Transport-Security
+ /: The X-Content-Type-Options header is not set. This could allow the user agent to render the content of the site in a different fashion to the MIME type. See: https://www.netsparker.com/web-vulnerability-scanner/vulnerabilities/missing-content-type-header/
+ No CGI Directories found (use '-C all' to force check all possible dirs)
+ OPTIONS: Allowed HTTP Methods: GET, HEAD, POST, PUT, DELETE, OPTIONS .
+ HTTP method ('Allow' Header): 'PUT' method could allow clients to save files on the web server.
+ HTTP method ('Allow' Header): 'DELETE' may allow clients to remove files on the web server.
+ /: Web Server returns a valid response with junk HTTP methods which may cause false positives.
```

Impact:

- Vulnerable to **Clickjacking** due to missing X-Frame-Options
- Browser might **downgrade security** or display content incorrectly due to missing X-Content-Type-Options
- Without HSTS, site does not enforce HTTPS, exposing it to **downgrade and MitM attacks**

Root Cause:

- Server is **not configured** to return essential HTTP security headers
- Outdated or default server configuration (Apache-Coyote/1.1)
- Lack of a secure deployment pipeline

Mitigation:

- Add X-Frame-Options: DENY or SAMEORIGIN to prevent Clickjacking
- Add X-Content-Type-Options: nosniff to enforce MIME type handling
- Add Strict-Transport-Security (HSTS) to force HTTPS
- Regularly review and apply secure headers based on industry standards (OWASP Secure Headers Project)

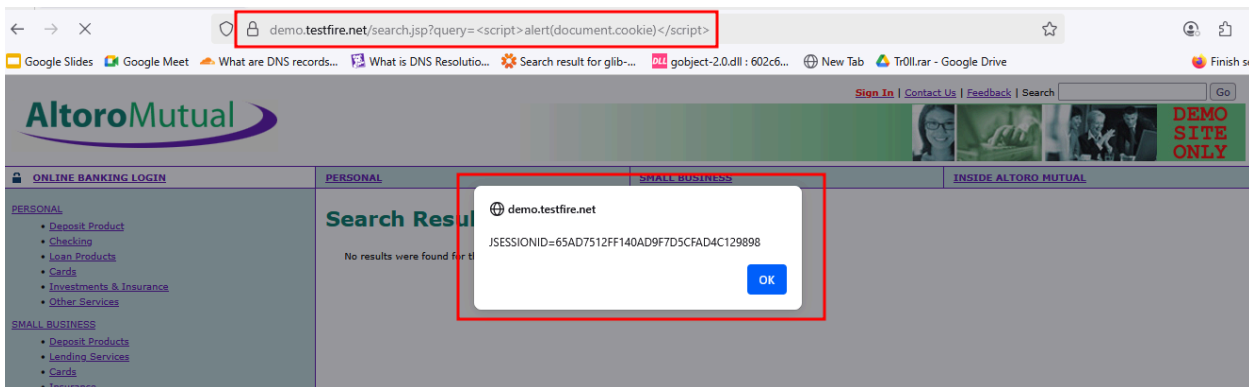
11. Session Cookie Exposure via document.cookie

Affect on

Infected URL	https://demo.testfire.net/search.jsp?query=%3Cscript%3Ealert(document.cookie)%3C/script%3E
Infected Parameter	Query
Parameter type	Get
Attack vector	An attacker crafts a malicious URL containing a JavaScript payload in the query parameter. When the victim visits the URL, the script executes in their browser due to improper input sanitization. The payload <code><script>alert(document.cookie)</script></code> successfully triggers a JavaScript alert displaying the user's session cookie. This confirms that the application is vulnerable to reflected XSS and that cookies are accessible via JavaScript.

Analysis

I was able to send a crafted input on the search field which in result led to a pop up alert displaying the session cookie.



Impact:

- Allows attackers to steal session cookies from users.
- Enables full session hijacking and impersonation of authenticated users.
- Can lead to further compromise such as unauthorized access, data theft, or privilege escalation.

Root Cause:

- Input from the query parameter is reflected into the HTML output without proper sanitization or encoding.
- The session cookie is not flagged as HttpOnly, allowing JavaScript access.

Mitigation:

- Apply context-aware output encoding for all user inputs reflected in the HTML.
- Set session cookies with the HttpOnly flag to prevent JavaScript access.
- Implement a Content Security Policy (CSP) to limit script execution.
- Use security libraries or frameworks that auto-escape untrusted input.

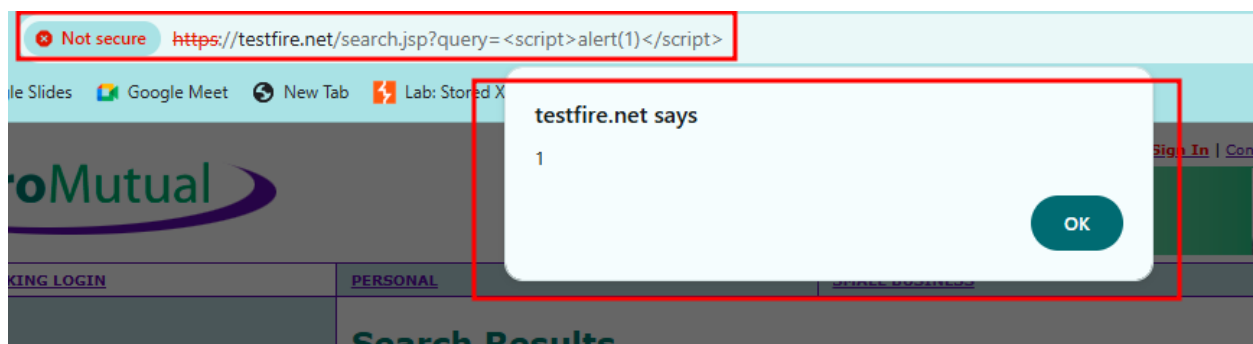
12. Reflected Cross Site Scripting

Affect on

Infected URL	https://testfire.net/search.jsp?query=<script>alert(1)</script>
Infected Parameter	Query
Parameter type	Get
Attack vector	<script>alert(1)</script>

Analysis

While testing the search.jsp page, a script payload was injected in the query parameter. Upon loading the crafted URL, a JavaScript alert popup appeared, confirming that the input was reflected and executed without sanitization. This validates the presence of a Reflected Cross-Site Scripting (XSS) vulnerability. An attacker could exploit this to execute malicious scripts in the user's browser.



Impact

- Execution of arbitrary JavaScript in the victim's browser
- Theft of session cookies or credentials
- Phishing or redirection to malicious sites
- Full control of user's browser session (if combined with session hijacking)

Root Cause

- User input in query parameter is reflected into the page without proper output encoding

- No HTML or JavaScript sanitization before displaying data on the search results page

Mitigation

- Use output encoding (e.g., escape <, >, ", ' characters) before reflecting input into HTML
- Use libraries like OWASP Java Encoder, ESAPI, or built-in framework functions
- Apply Content Security Policy (CSP) to block inline scripts
- Validate input to only allow safe characters (e.g., alphanumeric) if appropriate

13. Improper input validation

Affect On

Infected URL	http://testfire.net/secure/transferFunds.jsp
Infected Parameter	amount
Parameter type	post
Attack vector	The attacker intercepts the request using Burp Suite and modifies the amount value (e.g., from 100 to 100000) before forwarding it to the server. The backend accepts the manipulated amount without validation.

Analysis

The application allows users to modify the amount of money being transferred by intercepting and changing the request in Burp Suite. This indicates a lack of server-side input validation. The system blindly trusts the client-supplied data instead of validating it on the server. This can lead to unauthorized fund transfers, financial manipulation, and business logic abuse.

AL SMALL BUSINESS INSIDE ALTORO MUTUAL

Transfer Funds

From Account: 800000 Corporate ▼

To Account: 800001 Checking ▼

Amount to Transfer: 100

Transfer Money

© 2025 Altoro Mutual, Inc. This web application is open source! [Get your copy from GitHub](#) and take ad

```

1 POST /bank/doTransfer HTTP/1.1
2 Host: testfire.net
3 Cookie: JSESSIONID=14CE61153D6552233EEB203CF03FBE04;
  AltoroAccounts=
  ODAwMDAwfkNvcnBvcnFOZX4tMS4xODQONjcONDg3Nzk3MjYyRTIwfdGwMDAwMX5DaG
  Vja2luZ34yLjM2ODkzNDg4MTU2MDA5MjdFMjB8
4 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:140.0)
  Gecko/20100101 Firefox/140.0
5 Accept:
  text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
6 Accept-Language: en-US,en;q=0.5
7 Accept-Encoding: gzip, deflate, br
8 Content-Type: application/x-www-form-urlencoded
9 Content-Length: 78
10 Origin: https://testfire.net
11 Referer: https://testfire.net/bank/transfer.jsp
12 Upgrade-Insecure-Requests: 1
13 Sec-Fetch-Dest: document
14 Sec-Fetch-Mode: navigate
15 Sec-Fetch-Site: same-origin
16 Sec-Fetch-User: ?1
17 Priority: u=0, i
18 Te: trailers
19 Connection: keep-alive
20
21 fromAccount=800000&toAccount=800001&transferAmount=5000&transfer=
  Transfer+Money

```

MY ACCOUNT

PERSONAL

SMALL BUSINESS

INSIDE ALTORO MUTUAL

I WANT TO ...

- [View Account Summary](#)
- [View Recent Transactions](#)
- [Transfer Funds](#)
- [Search News Articles](#)
- [Customize Site Language](#)

ADMINISTRATION

- [Edit Users](#)

Transfer Funds

From Account:

To Account:

Amount to Transfer:

5000.0 was successfully transferred from Account 800000 into Account 800001 at 6/26/25 5:19 AM.

[Privacy Policy](#) |
 [Security Statement](#) |
 [Server Status Check](#) |
 [REST API](#) |
 © 2025

Impact

- Allows unauthorized manipulation of monetary transactions.
- Bypasses business logic controls such as transfer limits and balance checks.
- Enables users to transfer unrealistic or unauthorized amounts.
- Can lead to financial fraud and data integrity issues.
- May result in loss of user trust and legal consequences if exploited in production.

Root Cause

- The server trusts client-side input without verification.
- Absence of server-side validation for critical fields like transaction amount.
- No enforcement of business rules (e.g., transfer limits, account balance check) on the backend.
- Over-reliance on frontend validation which can be easily bypassed.

Mitigation

- Implement strict server-side validation of all transaction parameters, especially monetary values.
- Enforce business rules (such as minimum and maximum transfer limits) on the server side.
- Confirm account ownership and user authorization for every fund transfer.
- Use integer/decimal type checks and reject non-numeric or out-of-bound values.
- Log and monitor all financial transactions for anomalies and trigger alerts for suspicious activity.

14. Security misconfiguration – Missing Security Headers

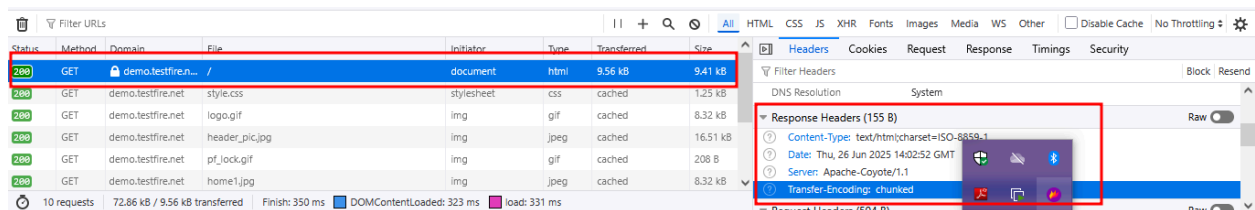
Affect on

Infected URL	https://demo.testfire.net/
Infected Parameter	<i>Not applicable</i> (This is a server misconfiguration, not input-based)
Parameter type	N/A
Attack vector	HTTP response lacks security headers that protect against browser-based threats like clickjacking, SSL stripping, and MIME-type sniffing.

Analysis

The web application at <https://demo.testfire.net> is missing critical HTTP security headers such as Strict-Transport-Security, X-Frame-Options, and X-Content-Type-Options. These headers are essential for defending against well-known browser-based attacks like clickjacking, SSL stripping, and MIME-type sniffing.

Although not an input-based vulnerability, the absence of these headers indicates weak server-side configuration and leaves the application open to exploitation through client-side behavior. Implementing these headers is a low-effort, high-impact security control that significantly reduces the attack surface.



Impact

- Allows clickjacking due to missing X-Frame-Options.
- Susceptible to SSL downgrade/MITM attacks without Strict-Transport-Security.
- Vulnerable to MIME-type sniffing without X-Content-Type-Options.

Root Cause

- Server is not configured to include essential security headers in HTTP responses.

Mitigation

- Add Strict-Transport-Security: max-age=31536000; includeSubDomains.
- Add X-Frame-Options: DENY.
- Add X-Content-Type-Options: nosniff.
- Implement headers via application filter or server configuration (Apache/Nginx/Tomcat).

15. Displaying internal server error (informational).

Affect on

Infected URL	https://testfire.net/index.jsp/conten=../../../../../etc/hosts
Infected Parameter	<i>Content</i>
Parameter type	Path parameter (part of the URL)
Attack vector	An attacker attempted to access system files using directory traversal sequences (../../../../../etc/hosts) through a path-based parameter.

Analysis

The server responded with an error (such as HTTP 500 Internal Server Error or 404 Not Found). The error page disclosed "Apache Tomcat", revealing backend server technology. No file contents were returned, but the response confirms the input was processed by the backend logic.

HTTP Status 500 – Internal Server Error

Type Exception Report

Message java.lang.NullPointerException

Description The server encountered an unexpected condition that prevented it from fulfilling the request.

Exception

```
org.apache.jasper.JasperException: java.lang.NullPointerException
    org.apache.jasper.servlet.JspServletWrapper.handleJspException(JspServletWrapper.java:594)
    org.apache.jasper.servlet.JspServletWrapper.service(JspServletWrapper.java:510)
    org.apache.jasper.servlet.JspServlet.serviceJspFile(JspServlet.java:395)
    org.apache.jasper.servlet.JspServlet.service(JspServlet.java:339)
    javax.servlet.http.HttpServlet.service(HttpServlet.java:731)
    org.apache.tomcat.websocket.server.WsFilter.doFilter(WsFilter.java:52)
```

Root Cause

java.lang.NullPointerException

Note The full stack trace of the root cause is available in the server logs.

Apache Tomcat/7.0.92

Impact:

- Confirms that the application accepts and processes unsafe file paths from user input.
- May lead to unauthorized access to system files if traversal is not blocked.
- Server error reveals internal behavior and underlying server (Apache Tomcat), aiding reconnaissance.

Root Cause:

- Lack of input validation or sanitization on file path parameters.
- Application logic likely resolves user-supplied paths without enforcing directory scope.
- No restriction on traversal sequences like ../.

Mitigation:

- Sanitize input to block directory traversal patterns (../, etc.).
- Implement allowlisting for file access.
- Use secure APIs that enforce base directory constraints.
- Suppress server error messages and configure custom error pages in production.

Conclusion

The security assessment of the application hosted at testfire.net identified a total of 15 distinct vulnerabilities affecting the overall security posture of the system. These findings include 2 critical, 8 high, 3 medium, and 2 low severity vulnerabilities. The application exhibits multiple weaknesses across all layers—ranging from input validation flaws and insecure design to cryptographic misconfigurations and insufficient hardening of authentication mechanisms.

Particularly concerning are the SQL Injection vulnerability enabling login bypass and the unrestricted internal content exposure, both categorized as critical. These issues pose an immediate risk to the confidentiality, integrity, and availability of the application and its data.

Several high-severity vulnerabilities—such as clear text credential transmission, exposed development tools, weak authentication design, and insecure direct object references—further increase the attack surface and indicate a lack of secure coding practices and defense-in-depth controls.

Additionally, common issues such as missing security headers, improper input validation, and XSS vulnerabilities demonstrate gaps in basic web application security hygiene.

While many of the issues are easily exploitable, suggesting a high risk of compromise, they can also be systematically remediated by following secure coding standards, implementing proper access control mechanisms, and adopting industry best practices for secure software design and deployment.